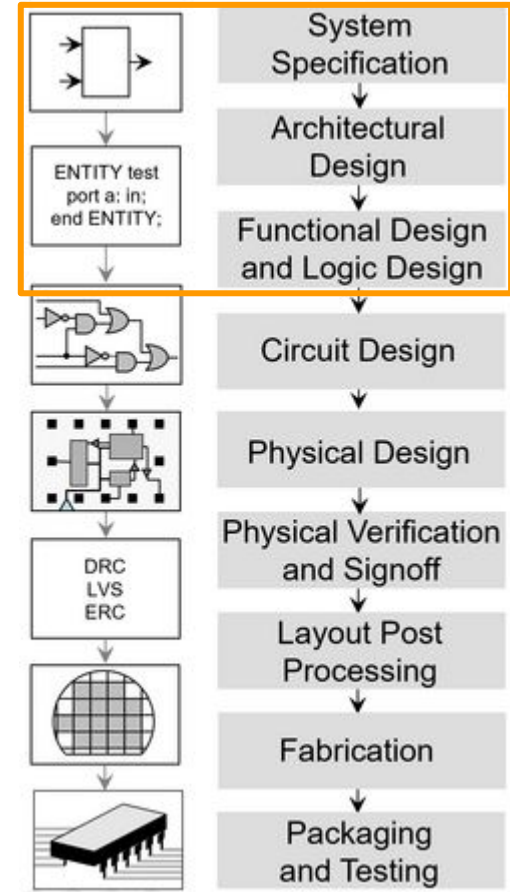


Why Verilog?

And more questions

Because we want to make chips

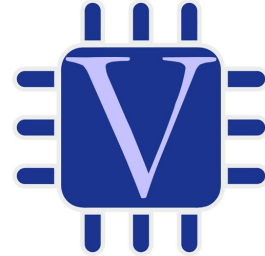
- You can't edit silicon like you can with code – errors at a later stage are significantly harder to fix
- Chip design is broken into stages:
 - * **Specification** → define what ("4-bit counter, sync reset, active-high enable")
 - * **High-Level Design** → define the pieces (ALU, decoder, registers) and their connections
 - * **Micro/Low-Level Design** → define how each piece works (FSMs, muxes, counters)
 - * **RTL Coding** → translate into Verilog/VHDL
 - * **Simulation** → verify correctness (~60-70% of project time)
 - **Synthesis** → RTL → gates
 - **Place & Route** → gates → physical layout → GDS file
 - **Post-Silicon Validation** → test the real chip
- *: Parts that we will touch upon
- [\[source\]](#)



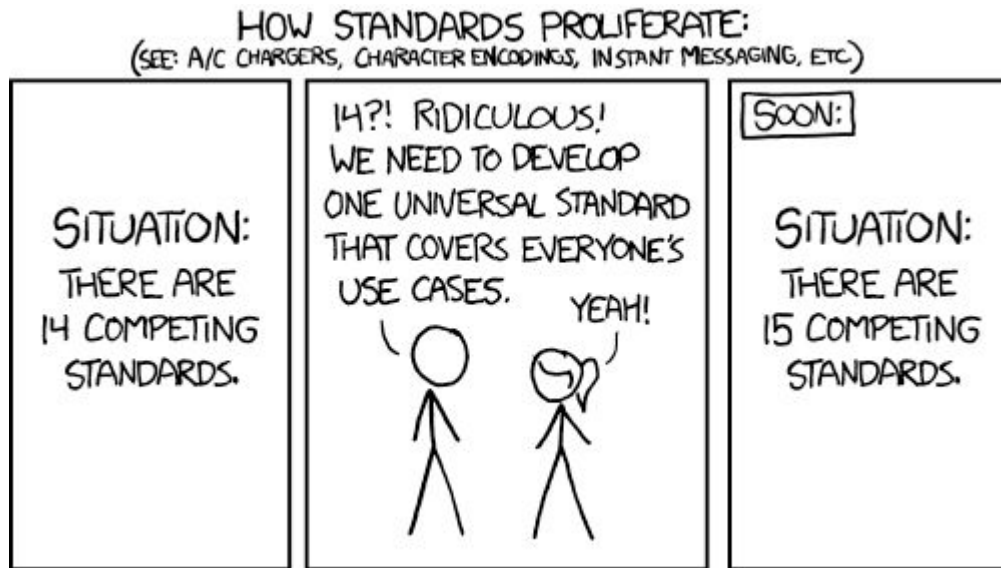
What are Hardware Description Languages

A hardware description language (HDL) is a specialized computer language used to describe the structure and behavior of electronic circuits, usually to design ASICs and to program FPGAs.

- Key difference from software languages: statements run concurrently by default (like real gates do), not line-by-line
- The same code can be:
 - **Simulated** → check correctness before anything is built
 - Synthesized → converted into an actual gate-level circuit
- Examples: Verilog, VHDL, SystemVerilog



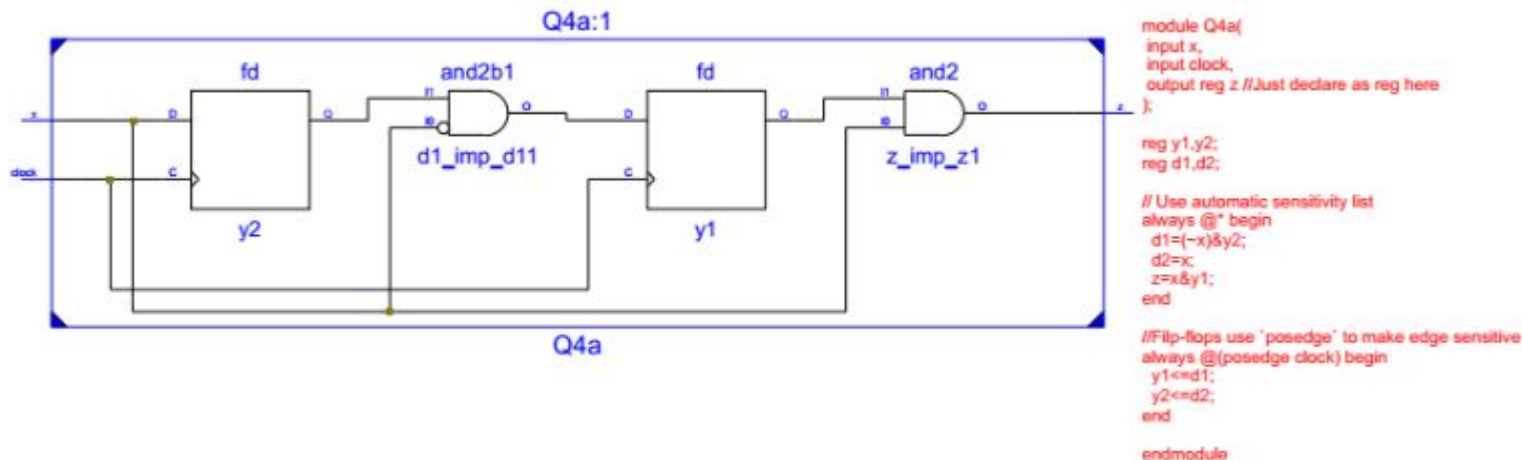
Why another language?



Why use Hardware Description Languages?

Relevant SO post: <https://electronics.stackexchange.com/q/601664>

- Because circuits are inherently **concurrent** (multiple components functioning **simultaneously**) – hard to implement in a lang. built for the single-threaded CPU model
- Because it abstracts away details we don't care about
- Because it makes simulation easier to approach



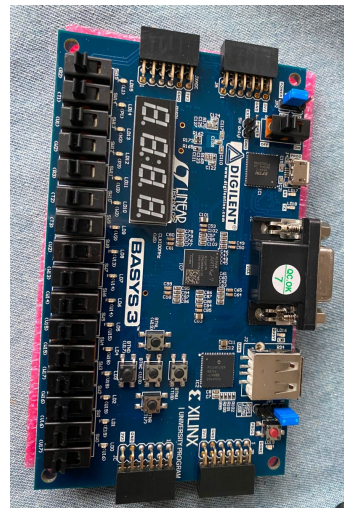
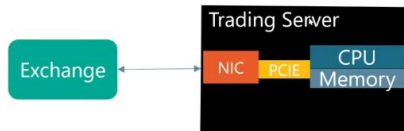
Why not more software?

Because it's too slow for high-frequency trading

- CPUs are **general purpose** and constrained to repeat the fetch -> decode -> execute loop
- OS network stack, device drivers all add **unavoidable (and unpredictable) delay**
- An **FPGA / ASIC** allows you to hardwire computation for ultra low latency, reliable execution
- Eg: Used in high-frequency regime trading for **nanosecond** scale processing times

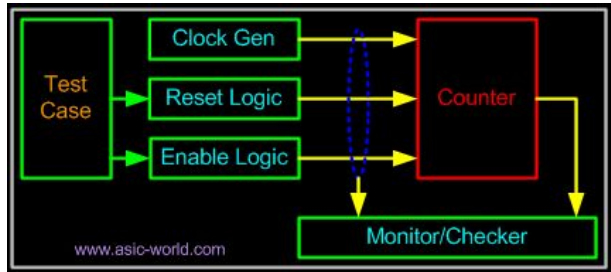
optiver 

LIMITATIONS TO SOFTWARE BASED TRADING



Again, why Verilog?

- **Easy to write** — describe complex behavior (registers, FSMs, arithmetic) in a few lines, instead of hand-drawing gates and wires one by one
- **Easy to test** — write testbenches to automatically drives inputs and checks outputs
- **Easy to see** — dump every signal to a waveform and watch your circuit's behavior visually
- **Synthesizable** (not covered) — converting the RTL to a real gate-level circuit on an FPGA/ASIC



```
1 module addbit (  
2     a      , // first input  
3     b      , // Second input  
4     ci     , // Carry input  
5     sum    , // sum output  
6     co     , // carry output  
7 );  
8 //Input declaration  
9 input a;  
10 input b;  
11 input ci;  
12 //Output declaration  
13 output sum;  
14 output co;  
15 //Port Data types  
16 wire a;  
17 wire b;  
18 wire ci;  
19 wire sum;  
20 wire co;  
21 //Code starts here  
22 assign {co,sum} = a + b + ci;  
23  
24 endmodule // End of Module addbit
```

Some definitions to help

For useful links and resources check the [repo](#).

- **Testbench** — code that isn't part of the real design; it just drives inputs into your circuit and checks outputs
- **Simulation** — running your circuit design virtually, before it's real, to check correctness
- **Waveform** — a timeline view of every signal in your circuit, used to visually debug behavior
- **Clock (clk)** — a repeating signal that ticks at a fixed rate
- **Module** — the basic building block in Verilog; a reusable, self-contained unit of hardware
- **Combinational logic** — output depends only on current inputs, no memory (e.g. an adder, a mux)
- **Sequential logic** — output depends on current inputs and past state (e.g. a counter, a register) — needs a clock
- **Register/Flip-flop** — a 1-bit memory element that holds its value until the next clock edge
- **Reset (rst)** — a signal that forces the circuit into a known starting state, ignoring everything else
- **Wire** — a signal that carries a value from one point to another; it doesn't store anything

Not relevant to verilog but mentioned previously:

- **RTL (Register Transfer Level)** — describing a circuit as data moving between registers through logic
- **FPGA (Field-Programmable Gate Array)** — a reprogrammable chip that can be configured based on HDL code
- **ASIC (Application-Specific Integrated Circuit)** — a chip custom-fabricated to do exactly one thing